第八讲 - 文件读写 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 文件与路径

3 安全打开与关闭文件

4 读取文本文件

5 文本清洗与数据转换

6 写入与追加结果

7 综合业务实践

8 总结与课后拓展

Python 程序设计基础

第8讲 文件读写 速记卡

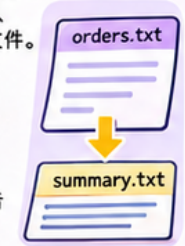
让程序学会读取数据、处理数据、保存结果！

Python
数据处理
电商实践

读取数据
处理数据
保存结果

1 为什么要学文件读写？

- 用一个简单的订单统计场景来说明：程序如何从文件读取数据、处理数据、并把结果保存成新文件。



- ✓ 订单数据需要长期保存
- ✓ 程序要读取 orders.txt
- ✓ 只统计“已完成”订单
- ✓ 计算订单数量和金额
- ✓ 生成新的 summary.txt 报告

订单数据（orders.txt）示例：

```
0001|杭州|360.00|已完成
0002|上海|640.00|已完成
0003|宁波|520.00|已退款
```

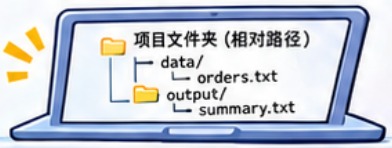
期望结果：

已完成订单 2笔，
已完成金额 1000.00元

2 文件与路径

- 文件 = 保存在电脑里的数据（如订单、报表、日志等）。
- 相对路径示例：data/orders.txt（项目下的 data 文件夹中的订单文件）。
- 输出路径示例：output/summary.txt（把统计结果保存到 output 文件夹）。
- 相对路径便于移动项目（换一台电脑也能用）。
- 可用 Path 检查文件是否存在。

```
from pathlib import Path
file_path = Path("data") / "orders.txt"
file_path.exists()  # 返回 True 或 False
```



安全打开与关闭文件

- ✓ 用 with open(...) 更安全、推荐使用。
- ✓ 离开代码块后，文件会自动关闭（不用再手动 file.close()）。
- ✓ 打开文件时需要说明：路径、模式、编码。

```
with open("data/orders.txt", "r",
          encoding="utf-8") as file:
    content = file.read()
```

常用的文件模式：

r	读取：打开文件并读取内容（最常用）。
w	重写：写入文件，会覆盖原有内容！
a	追加：在文件末尾追加内容，不会覆盖。

读取文本文件

- Python 提供多种读取文本的方式，常用方法：

读取方法	作用（通俗理解）
read()	一次读全部：把整个文件内容读成一个字符串。
readline()	读下一行：每次读取一行文本。
readlines()	得到行列表：把所有行读成一个列表。
for line in file	逐行处理：循环读取每一行（常用、推荐）。

订单记录通常适合逐行处理：一行就是一笔订单，便于解析和筛选。

```
for line in file:
    record = line.strip()  # 去掉行首尾的空白
    if not record:
        continue       # 跳过空行
```

5 文本清洗与数据转换

- 把从文件读到的“原始文本”，一步步变成可用的结构化数据（如订单号、城市、金额、状态）。

原始文本（一行订单） → strip() 去空白 → split("|") 按|分割 → 命名字段 更清晰 → float() 转金额（字符串→数字）

```
record = "0001|杭州|360.00|已完成"
fields = record.split("|")
order_id, city, amount_text, status = fields
amount = float(amount_text)
```

转换成更清晰的数据（字典示例）：

```
{"订单号": "0001",
 "城市": "杭州",
 "金额": 360.0,
 "状态": "已完成"}
```

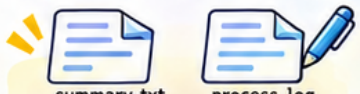


6 写入与追加结果

- ✓ write()：向文件写入一个字符串。
- ✓ print(..., file=file)：也能把内容写入文件。
- ✓ w 模式：重写文件（会覆盖原内容）。适合生成新的报告（如 summary.txt）。
- ✓ a 模式：追加内容（不覆盖）。适合记录日志（如 process.log）。

```
with open("output/summary.txt", "w",
          encoding="utf-8") as file:
    print("订单统计报告", file=file)
```

```
with open("output/process.log", "a",
          encoding="utf-8") as file:
    file.write("订单统计完成\n")
```



7 综合业务实践：订单统计报告

- 把所学知识串起来，完成一个完整的订单统计小项目！

业务流程（6 步）：

1. 读取订单文件
2. 解析每行订单
3. 只保留“已完成”订单
4. 汇总数量与金额
5. 写入 summary.txt
6. 重新读取并核验结果

关键函数（示例）：

- parse_order(line) → 解析一行订单，返回字典。
- read_orders(file_path) → 读取订单文件，得到订单列表。
- summarize_orders(orders) → 汇总“已完成”订单的数量和金额。
- write_report(file_path, summary) → 把统计结果写入文件。



运行结果（summary.txt）：

订单统计报告
已完成订单：2笔
已完成金额：1000.00元

学习提醒 / 易错点

- ✓ **当前工作目录不对，路径会报错：** 确保程序运行时的目录能找到你的文件。
- ✓ **编码通常用 UTF-8：** 如 encoding="utf-8"，可避免中文乱码。
- ✓ **w 会覆盖原内容，别乱用：** 需要保留原内容时，使用 a 模式追加。
- ✓ **文件读到的数据常常是字符串：** 如金额需要用 float() 转成数字后再计算。
- ✓ **写完文件后，最好重新读取检查结果：** 确认文件内容是否正确。
- ✓ **先想清楚：输入文件、处理规则、输出文件：** 理清流程，写代码更顺畅。



小提示：
多动手练一练，
从处理一份小数据开始，
你也能用 Python
完成实用的数据任务！

记住：文件读写不是背语法，而是把“外部数据 → 程序处理 → 结果保存”连成完整流程。

订单数据需要长期保存

某网店每天导出一份订单文件。运营人员需要统计已完成订单的数量和金额，并将统计结果保存为新的报告。

输入文件：orders.txt

```
0001|杭州|360.00|已完成
0002|上海|640.00|已完成
0003|宁波|520.00|已退款
```

运营人员需要：

- 读取全部订单记录；
- 只统计已完成订单；
- 计算订单数量和金额；
- 保存统计报告。

为什么需要文件

Notebook中的变量通常会随内核关闭而消失。文件可以长期保存数据，也能在程序、人员和系统之间传递数据。

订单统计任务的处理流程

1. 找到输入文件 `data/orders.txt`；
2. 使用UTF-8编码读取文件；
3. 将每行文本转换为订单数据；
4. 统计已完成订单的数量和金额；
5. 将结果写入 `output/summary.txt`；
6. 重新读取报告并核对结果。

期望生成的报告

订单统计报告

已完成订单：2笔
已完成金额：1000.00元

本讲关注的问题

文件在哪里？如何安全地读取和写入？怎样避免覆盖原始数据？如何检查输出结果？

文件处理的核心：读取外部数据，完成业务计算，保存处理结果。

程序如何使用文件

`orders.txt` ⟶ 读取数据 ⟶ 处理数据 ⟶ 写入结果 ⟶ `summary.txt`

文本文件

使用字符保存数据，打开后可以直接阅读。

- 常见格式包括TXT、CSV和JSON；
- 需要使用编码解释中文等字符；
- 本讲重点处理文本文件。

二进制文件

使用字节保存数据，通常需要专门的软件或程序解释。

- 常见格式包括图片、音频和PDF；
- Python读取后得到字节数据；
- 本讲只作简要了解。

文件操作需要明确的信息

程序需要知道文件的位置、操作方式和文本编码。后续将分别使用路径、打开模式和 `encoding` 参数说明这些信息。

相对路径与项目结构

项目文件夹

```
python-course/
  main.ipynb
  data/
    orders.txt
  output/
    summary.txt
```

假设当前工作目录是 `python-course`，读取订单文件时可以使用：

```
with open("data/orders.txt", "r",
↪   encoding="utf-8") as file:
  content = file.read()
```

- `data` 表示当前目录中的子文件夹；
- `orders.txt` 是要读取的文件；
- 相对路径的起点是当前工作目录。

相对路径的优点

只要项目内部的文件夹结构保持不变，整个项目移动到其他位置后，代码不需要修改。

路径问题的检查方法

相对路径

`data/orders.txt`

- 从当前工作目录开始查找；
- 代码较短，便于移动项目；
- 工作目录改变后可能失效。

绝对路径

`C:/python-course/data/orders.txt`

- 从文件系统的根目录开始；
- 文件位置明确；
- 换一台计算机后通常需要修改。

Python标准库中的 `Path` 可以检查路径：

```
from pathlib import Path
# 打印当前工作目录
print(Path.cwd())
# 用 / 运算符把两段路径拼起来，
↪  得到相对路径
file_path = Path("data") /
↪  "orders.txt"
print(file_path)
print(file_path.exists())
```

文件未找到时的检查顺序

1. 查看当前工作目录；
2. 检查文件夹和文件名写对了没；
3. 检查文件所在的层级；
4. 使用 `exists()` 确认文件是否存在。

安全打开文件

```python
with open("data/orders.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)

print(file.closed)   # True
```

打开文件所需的信息

- `data/orders.txt` 指定文件路径；
- `"r"` 表示读取文件；
- `"utf-8"` 用于正确解释中文。

文件对象

`open()` 打开文件并返回一个文件对象。

`as file` 将该对象保存到变量 `file` 中，后续通过它读取或写入数据。

为什么使用with语句

只有缩进的代码能够操作当前文件。离开代码块后，Python会自动关闭文件。 即使代码块运行时发生异常，也不需要手动调用 `close()` 。

三种常用的打开模式

模式	主要用途	文件不存在	文件已经存在
r	读取文件	产生错误	保留原有内容
w	重新写入文件	创建新文件	清空原内容后写入
a	追加内容	创建新文件	在文件末尾继续写入

```python
# 重新生成统计报告
with open("output/summary.txt", "w", encoding="utf-8") as file:
  file.write("已完成订单：2笔\n")

# 保留原有日志，追加一条记录
with open("output/process.log", "a", encoding="utf-8") as file:
  file.write("订单统计完成\n")
```

写入前必须确认模式

生成一份新的报告时使用 `w`，保留历史记录并继续添加时使用 `a`。不要使用 `w` 直接打开仍需保留的原始业务数据。

一次读取全部内容

```python
with open("data/orders.txt", "r", encoding="utf-8") as file:
    content = file.read()

print(content)
print(type(content))  # <class 'str'>
```

读取结果

`read()` 一次读取文件的全部内容，并返回一个字符串。文件中的换行符也会保留在字符串中。

适用场景

适合读取内容较少的说明文件、配置文件和短报告。文件较大时，全部内容会同时进入内存。

再次读取同一文件

`read()` 读取到文件末尾后，继续对同一个文件对象调用 `read()`，通常会得到空字符串。需要重新读取时，可以再次使用 `with open(...)` 打开文件。

逐行读取订单记录

```
with open("data/orders.txt", "r", encoding="utf-8") as file:
  for line in file:
    record = line.strip()
    if not record:
      continue

    print(record)
```

每次处理一行

文件对象可以直接用于 `for` 循环。循环每执行一次，变量 `line` 保存当前读取的一行文本。

清理行末换行符

`strip()` 返回一个新字符串，清除字符串两端的空格、换行符和其他空白字符。

逐行读取的优势

程序不需要一次把整个文件读入内存。订单记录、销售明细和日志通常适合逐行处理。

读取方法的选择

读取方式	返回结果	内存使用	适用场景
`read()`	一个字符串	一次保存全部内容	读取较小的完整文本
`readline()`	下一行字符串	只保存当前结果	明确需要读取下一行
`readlines()`	字符串组成的列表	一次保存全部行	需要对行列表进行操作
`for line in file`	依次得到每一行	每次处理一行	逐条处理业务记录

本讲的选择原则

较小文件需要整体内容时使用 `read()`。订单、交易明细和日志等记录型文件优先直接遍历文件对象。`readline()` 和 `readlines()` 在特定需要出现时再使用。

方法名相似但结果不同

`read()` 和 `readline()` 返回字符串，`readlines()` 返回列表。处理数据前应先确认返回值的类型。

清理一行原始文本

```
with open("data/orders.txt", "r", encoding="utf-8") as file:
  line = file.readline()

print(repr(line)) # '0001|杭州|360.00|已完成\n'

record = line.strip()

print(record) # 0001|杭州|360.00|已完成
```

查看不可见字符

直接输出文本时，换行符不容易观察。`repr()` 可以显示字符串中的 `\n` 等特殊字符。

清理字符串

`strip()` 清除字符串两端的空格、换行符和其他空白字符，并返回一个新的字符串。

清洗数据的目的

文件中的原始文本常带有换行符或多余空格。先清理文本，可以减少后续比较、拆分和类型转换中的错误。

拆分订单字段

```
record = "0001|杭州|360.00|已完成"

fields = record.split("|")
print(fields) # ['0001', '杭州', '360.00', '已完成']

order_id, city, amount_text, status = fields

print(order_id)
print(amount_text)
```

拆分字符串

`split("|")` 以竖线为分隔符拆分字符串，返回一个由多个字符串组成的列表。

为字段命名

将四个字段分别保存到四个变量后，变量名能够直接说明每个字段的业务含义。

字段数量必须匹配

四个变量需要对应四个字段。真实业务文件可能存在缺少字段或分隔符错误，处理前应先确认文件格式。

从文本到订单字典

```
def parse_order(line):
  order_id, city, amount_text, status = (line.strip().split("|"))
  return {"订单号": order_id,"城市": city, "金额": float(amount_text),
  ↪  "状态": status}

order = parse_order("0001|杭州|360.00|已完成\n")

print(order)
```

类型转换

文件读取的字段都是字符串。订单号、城市和状态继续使用字符串，金额使用 `float()` 转换为数值。

字典表示业务记录

字典用键说明字段含义。后续可以通过 `order["金额"]` 和 `order["状态"]` 完成统计。

写入新的统计报告

```
summary = {
"已完成订单": 2,
"已完成金额": 1000.0
}

with open("output/summary.txt", "w", encoding="utf-8") as file:
  file.write("订单统计报告\n")
  file.write(f"已完成订单: {summary['已完成订单']}笔\n")
  file.write(f"已完成金额: {summary['已完成金额']:.2f}元\n")
```

写入字符串

`write()` 接收字符串。代码使用f-string将订单数量和金额转换为报告文本。

手动添加换行

`write()` 不会自动换行。需要在字符串末尾加入 `\n` 。

写入模式会覆盖原内容

每次使用 `w` 模式打开 `summary.txt` ，Python都会清空文件原有内容，再写入本次生成的报告。

追加运行日志

```
message = "订单统计完成：共处理3条记录\n"

with open("output/process.log", "a", encoding="utf-8") as file:
  file.write(message)
```

如果程序运行两次，日志文件包含：

日志文件内容

订单统计完成：共处理3条记录
订单统计完成：共处理3条记录

追加模式的作用

`a` 模式保留原有内容，并把新内容写到文件末尾。文件不存在时，Python会创建新文件。

Notebook中的重复运行

重复运行同一个代码单元会重复追加内容。实验前应确认日志是否需要保留或重新生成。

模式取决于业务目的

统计报告通常需要重新生成，因此使用 `w` 。运行日志需要保留历史记录，因此使用 `a` 。

生成并核验报告

```python
completed_count = 2
completed_amount = 1000.0

with open("output/summary.txt", "w", encoding="utf-8") as file:
  print("订单统计报告", file=file)
  print(f"已完成订单：{completed_count}笔", file=file)
  print(f"已完成金额：{completed_amount:.2f}元", file=file)

with open("output/summary.txt", "r", encoding="utf-8") as file:
  report = file.read()

print(report)
print("已完成订单：2笔" in report)  # True
```

使用print写入文件

为 `print()` 指定 `file` 参数后，输出内容会进入文件。每次调用默认添加换行符。

写入后重新读取

重新打开报告可以检查文件是否生成，并核对关键文字和业务数据是否正确。

文件任务需要闭环：程序成功运行不等于结果正确。完成写入后，还应检查文件位置、文件内容和关键统计值。

订单统计报告任务

输入文件

```
0001|杭州|360.00|已完成
0002|上海|640.00|已完成
0003|宁波|520.00|已退款
```

期望生成的报告

```
订单统计报告
已完成订单：2笔
已完成金额：1000.00元
```

业务规则

- 只统计状态为“已完成”的订单；
- 计算订单数量和订单金额；
- 不修改原始订单文件。

程序任务分解

1. 读取订单文件；
2. 将文本转换为订单字典；
3. 汇总已完成订单；
4. 写入统计报告；
5. 重新读取并核验结果。

读取并转换订单记录

```
def parse_order(line):
  order_id, city, amount_text, status = line.strip().split("|")
  return {"订单号": order_id, "城市": city, "金额": float(amount_text),
  ↪  "状态": status}

def read_orders(file_path):
  orders = []
  with open(file_path, "r", encoding="utf-8") as file:
    for line in file:
      if line.strip():
        orders.append(parse_order(line))
  return orders
```

单行解析函数

`parse_order()` 只负责处理一行文本，并返回一个订单字典。

文件读取函数

`read_orders()` 逐行读取文件，忽略空行，并将所有订单字典保存到列表中。

统计已完成订单

```python
def summarize_orders(orders):
    completed_count = 0
    completed_amount = 0.0

    for order in orders:
        if order["状态"] == "已完成":
            completed_count += 1
            completed_amount += order["金额"]

    return {"已完成订单": completed_count, "已完成金额": completed_amount}

orders = read_orders("data/orders.txt")
summary = summarize_orders(orders)

# {'已完成订单': 2, '已完成金额': 1000.0}
print(summary)
```

筛选业务记录

条件语句只保留状态为“已完成”的订单，退款订单不会进入统计结果。

返回汇总数据

函数返回一个字典，统一保存订单数量和金额，方便后续输出报告。

写入并核验统计报告

```
def write_report(file_path, summary):
  with open(file_path, "w",encoding="utf-8") as f:
      print("订单统计报告", file=file)
      print(f"已完成订单: {summary['已完成订单']}笔", file=f)
      print(f"已完成金额: {summary['已完成金额']:.2f}元", file=f)

orders = read_orders("data/orders.txt")
summary = summarize_orders(orders)
write_report("output/summary.txt", summary)

with open("output/summary.txt", "r", encoding="utf-8") as f:
  report = f.read()

print(report)
```

函数组成完整流程

`read_orders()` 负责输入，`summarize_orders()` 负责业务计算，`write_report()` 负责输出。每个函数只承担一个明确任务。

最终核验：确认报告文件位于 `output` 文件夹，并核对“2笔”和“1000.00元”是否与人工计算结果一致。

本讲总结

确定路径 $\longrightarrow$ 安全打开 $\longrightarrow$ 读取文本 $\longrightarrow$ 转换数据 $\longrightarrow$ 业务计算 $\longrightarrow$ 写入并核验

核心方法

- 使用相对路径组织项目文件；
- 使用 `with open(...)` 管理文件；
- 使用 `read()` 或逐行遍历读取；
- 使用 `write()` 或 `print()` 写入。

需要重点检查

- 当前工作目录是否正确；
- 文本编码是否为UTF-8；
- `w` 是否会覆盖重要文件；
- 字符串是否完成类型转换；
- 输出结果是否经过核验。

文件处理的基本原则

先明确输入文件、业务规则和输出要求，再编写代码。程序成功运行后，还需要检查文件位置和业务结果。

课后实践

增加两条订单记录

```
0004|杭州|120.00|已完成
0005|上海|200.00|已退款
```

实践任务

1. 将两条记录追加到 `data/orders.txt`；
2. 重新运行订单统计程序；
3. 将报告写入 `output/summary.txt`；
4. 将“本次共处理5条记录”追加到日志；
5. 重新读取报告和日志，检查结果。

期望统计结果

已完成订单应为3笔，已完成金额应为1120.00元。

进一步练习

增加“按城市统计已完成金额”的功能，并在报告中分别输出杭州和上海的结果。

其他读取与写入方法

```python
with open("data/orders.txt", "r", encoding="utf-8") as file:
  first_line = file.readline()
  remaining_lines = file.readlines()

print(first_line)
print(type(remaining_lines))  # list

lines = ["订单统计报告\n",
"已完成订单：2笔\n",
"已完成金额：1000.00元\n"]

with open("output/summary.txt", "w", encoding="utf-8") as file:
  file.writelines(lines)
```

读取多行

`readline()` 读取下一行，`readlines()` 读取剩余所有行并返回列表。较大文件不宜一次读取全部行。

批量写入

`writelines()` 依次写入多个字符串，但不会自动添加换行符。列表中的每个字符串应自行包含 `\n`。

使用路径对象读写文件

```python
from pathlib import Path

data_path = Path("data") / "orders.txt"
output_path = Path("output") / "summary.txt"

print(data_path.exists())
print(data_path.name)
print(data_path.parent)

content = data_path.read_text(encoding="utf-8")
output_path.write_text("订单统计报告\n已完成订单：2笔\n", encoding="utf-8")
```

路径对象的作用

`Path` 可以拼接路径、检查文件是否存在，并获取文件名和父文件夹。代码不需要手动处理不同操作系统的路径分隔符。

便捷读写方法

`read_text()` 和 `write_text()` 适合较小的文本文件。它们会一次读取或写入全部内容，较大文件仍应使用 `with open(...)` 逐行处理。

读取CSV订单文件

```
import csv

completed_count = 0
completed_amount = 0.0

with open("data/orders.csv", "r", encoding="utf-8-sig", newline="") as
↪  file:
  reader = csv.DictReader(file)

for order in reader:
  if order["状态"] == "已完成":
    completed_count += 1
    completed_amount += float(order["金额"])

print(completed_count, completed_amount)
```

CSV文件的表头和csv模块

文件第一行可以是 `订单号,城市,金额,状态` 。 `DictReader` 使用表头作为字典的键。真实CSV字段可能包含逗号、引号和换行符。直接使用 `split(",")` 容易错误拆分数据。 `utf-8-sig` 还可以读取部分表格软件导出的带BOM文件。

金额数据的精确计算

```python
print(0.1 + 0.2) # 0.30000000000000004

from decimal import Decimal

amount_1 = Decimal("0.10")
amount_2 = Decimal("0.20")
print(amount_1 + amount_2) # 0.30

unit_price_cents = 1990
quantity = 3
total_cents = unit_price_cents * quantity
print(total_cents) # 5970
```

关于金额数据

课堂示例使用 `float` 练习类型转换。对精度要求严格的结算业务通常使用 `Decimal` 或以“分”为单位的整数。

使用要点

创建 `Decimal` 金额时，应优先传入字符串，例如 `Decimal("19.90")` 。使用整数分时，19.90元可以保存为1990分。

二进制文件与文件位置

```python
with open("img/logo.png", "rb") as file:
  header = file.read(8)
  position = file.tell()

print(header)
print(position)  # 8

file.seek(0)
first_byte = file.read(1)

print(first_byte)
```

二进制模式

`rb` 以二进制方式读取文件，返回 `bytes` 数据，不使用文本编码。

文件位置

`tell()` 返回当前位置，`seek(0)` 将位置移动到文件开头。

使用范围

图片、音频和压缩文件常使用二进制模式。需要精确按字节定位时，应优先在二进制模式下使用 `seek()` 和 `tell()` 。

THE END